Directives

Concept

Directives are classes that add behavior to elements in the DOM.

Key Point: Components ARE directives with templates. Every component is technically a directive, but directives without templates are more lightweight and focused on DOM manipulation.

Types of Directives

Type	Description	Example	Asterisk Required
Structural	Modify the DOM structure by adding/removing elements	`*ngIf`, `*ngFor`, `*ngSwitch`	Yes (in template)
Attribute	Modify the appearance or behavior of existing elements	`ngClass`, `ngStyle`, `ngModel`	No
Component	Directives with templates (most common)	Custom components	No

Structural Directives

Characteristics

- **Asterisk (`*`)**: Required when applied in templates
- **Purpose**: Add, remove, or manipulate DOM elements

Key Differences

Aspect	Structural	Attribute	Component
Template	No	No	Yes
DOM Manipulation	Add/Remove elements	Modify existing	Full component
Asterisk	Required	Not required	Not required
Use Case	Layout changes	Behavior/Appearance	Reusable UI blocks

Best Practices

1. **Use structural directives** for conditional rendering and loops
2. **Use attribute directives** for styling and behavior modifications
3. **Use components** for reusable UI blocks with templates
4. **Track by function** in `*ngFor` for performance
5. **Avoid** creating directives when a component would be more appropriate
6. **Keep directives focused** on a single responsibility

Components

Components are directives with templates that define views, encapsulate logic, and manage data. Components combine HTML templates, TypeScript classes, and CSS styles to create reusable UI elements.

Component Metadata

Property	Description	When to Use
`selector`	CSS selector for the component	Always required
`templateUrl`	Path to HTML template file	For external templates
`template`	Inline HTML template	For small templates
`styleUrls`	Array of CSS file paths	For external styles
`styles`	Inline CSS styles	For small styles
`standalone`	Makes component standalone	Angular 14+ (recommended)
`imports`	Modules/components to import	Standalone components
`changeDetection`	Change detection strategy	Performance optimization

Standalone Components (Angular 14+)

Concept

Standalone components don't require NgModules. They can directly import what they need, making the architecture more modular and tree-shakeable.

When to Use

- **New projects**: Default approach in Angular 14+
- **Micro frontends**: Better for module federation
- **Lazy loading**: Easier to lazy load individual components
- **Tree shaking**: Better bundle optimization

Component Communication

Method	Direction	Use Case
`@Input()`	Parent → Child	Pass data down
`@Output()`	Child → Parent	Emit events up
Services	Any → Any	Shared state/logic
ViewChild	Parent → Child	Access child component
Content Projection	Parent → Child	Project content

Lifecycle Hooks

Concept

Methods that Angular calls at specific points in a component's or directive's lifecycle. They allows us to tap into key moments to perform custom logic.

Lifecycle Hook Order

Hook	Timing	Purpose	Common Use Cases
`ngOnChanges`	Before `ngOnInit`, when inputs change	React to input changes	Validate inputs, update derived state
`ngOnInit`	Once, after first `ngOnChanges`	Component initialization	Fetch data, setup subscriptions
`ngDoCheck`	Every change detection cycle	Custom change detection	Custom change detection logic
`ngAfterContentInit`	After content projection	After projected content initialized	Access projected content
`ngAfterContentChecked`	After every content check	After projected content checked	React to projected content changes
`ngAfterViewInit`	After view initialized	After view and child views initialized	Access DOM, child components
`ngAfterViewChecked`	After every view check	After view and child views checked	React to view changes
`ngOnDestroy`	Before component destruction	Cleanup	Unsubscribe, clear timers, cleanup

Change Detection

Change Detection is Angular's mechanism to detect changes in the application state and update the DOM accordingly.

Change Detection Strategies

Strategy	Description	When Triggered	Performance
Default	Checks component and all children on every change detection cycle	Every async event, timer, HTTP request	Slower
OnPush	Only checks when inputs change, events occur, or async pipe emits	Input reference changes, DOM events, async pipe	Faster
None	Disables change detection	Never (manual only)	Fastest (but dangerous)

Performance Impact

Strategy	Change Detection Frequency	Performance Impact
Default	Every cycle, all components	High
OnPush	Only when inputs/events change	Low
None	Never (manual only)	None (but risky)

What Triggers Change Detection?

Event	Default Strategy	OnPush Strategy
User click	✅	✅
HTTP request completes	✅	❌ (unless async pipe)
setTimeout/setInterval	✅	❌
Promise resolves	✅	❌ (unless async pipe)
Observable emits	✅	❌ (unless async pipe)
Input reference changes	✅	✅
Input property changes	✅	❌

Manual Change Detection

markForCheck()

Marks the component and all ancestors for check in the next change detection cycle.

When to Use:

- With OnPush strategy
- When you need to trigger change detection manually
- After async operations in OnPush components

detectChanges()

Immediately runs change detection on the component and its children.

When to Use:

- Testing scenarios
- Third-party library integration
- Specific edge cases

Differences: markForCheck vs detectChanges

Aspect	markForCheck	detectChanges
Timing	Next change detection cycle	Immediately
Scope	Component + ancestors	Component + children
Performance	Better (batched)	Can cause issues if overused
Use Case	OnPush components	Testing, edge cases

View Encapsulation

View Encapsulation controls how Angular handles component styles and DOM isolation. It controls whether styles are scoped to the component or applied globally, and how the component's DOM is isolated from the rest of the application.

Encapsulation Strategies

Strategy	Style Isolation	DOM Isolation	Shadow DOM	Browser Support	Performance
Emulated (Default)	✅ Yes	❌ No	❌ No	✅ All	✅ Good
None	❌ No	❌ No	❌ No	✅ All	✅ Best
ShadowDom	✅ Yes	✅ Yes	✅ Yes	⚠️ Limited	⚠️ Slower

Key Differences

Feature	Emulated	None	ShadowDom
Style Scoping	✅ Yes	❌ No	✅ Yes
DOM Isolation	❌ No	❌ No	✅ Yes
JS Isolation	❌ No	❌ No	✅ Yes
ng-deep Works	✅ Yes	N/A	❌ No
Browser Support	✅ All	✅ All	⚠️ Modern
Performance	✅ Good	✅ Best	⚠️ Slower

ng-deep (Deprecated)

Concept

`::ng-deep` (or `/deep/` , `>>>`) allows styles to pierce through view encapsulation.

Best Practices

1. Use Emulated (Default)

2. Use None for Shared Styles

3. Avoid ng-deep

4. Use CSS Custom Properties

Performance Considerations

Strategy	Style Processing	DOM Manipulation	Overall
Emulated	Medium	Low	✅ Good
None	Low	Low	✅ Best
ShadowDom	High	High	⚠️ Slower

Dependency Injection

Dependency Injection is a design pattern where dependencies are provided to a class rather than the class creating them itself.

Injection Tree Hierarchy

Angular provides dependencies at different levels in the application hierarchy:

```
Platform Injector
  └── Application Injector (Root)
        └── Module Injectors
              └── Component Injectors
```

Injection Levels

Level	How to Provide	Scope	Use Case
Platform	`platformBrowser().bootstrapApplication()`	Entire platform	Micro frontends, platform-wide services
Application (Root)	`providedIn: 'root'`	Entire application	Singleton services, shared services
Module	`providers: []` in NgModule	Module and children	Module-specific services
Component	`providers: []` in Component	Component and children	Component-specific instances

Root-Level Injection (Recommended)

DI Resolution Order

1. Check current injector
2. Check parent injector
3. Continue up the tree
4. Throw error if not found (unless `@Optional()`)

Default Choice: Use `providedIn: 'root'` for 95% of your services.

Component Interaction

Components in Angular applications need to communicate with each other.

Communication Patterns

Pattern	Direction	Use Case	Complexity
@Input()	Parent → Child	Pass data down	Low
@Output()	Child → Parent	Emit events up	Low
Services	Any → Any	Shared state/logic	Medium
ViewChild	Parent → Child	Access child component	Medium
Content Projection	Parent → Child	Project content	Low

Best Practices

1. Prefer @Input() and @Output()

2. Use Services for Complex State

3. Use Content Projection for Flexible Layouts

4. Use Async Pipe with Services

5. Type Your Inputs and Outputs

NgRx

state management library for Angular applications based on Redux. It provides a centralized state container using RxJS observables, making state changes predictable, traceable, and testable.

Core Concepts

Concept	What It Is	Purpose	Example
Actions	Events describing what happened	Trigger state changes	`loadUsers()`, `addUser()`
Reducers	Pure functions that update state	Define how state changes	`userReducer(state, action)`
Store	Single source of truth for app state	Holds all application state	`Store<AppState>`
Effects	Handle side effects (API calls, etc.)	Perform async operations	HTTP calls, localStorage
Selectors	Functions to select/derive data from store	Get data from store efficiently	`selectAllUsers`

Lifecycle

Step	What Happens	Between Steps
1. **Action Dispatched**	Component calls `store.dispatch(action)`	Action sent to store
2. **Reducer**	Pure function receives action, returns new state	State object replaced (immutable)
3. **State Updated**	Store updates with new state	Selectors recompute if needed
4. **Effect Triggered**	Effect listens via `ofType()`, matches action	Effect performs async operation
5. **Effect Dispatches**	Effect dispatches success/failure action	New action goes back to step 2
6. **Components Update**	Selectors emit new values, components re-render	UI reflects new state

Pipes

functions that transform data in Angular templates. Pipes are pure by default, meaning they only recompute when input values change.

Pipe Types

Type	Description	Performance	When to Use
Pure	Only recomputes when input reference changes	✅ Fast	Default, most pipes
Impure	Runs on every change detection cycle	⚠️ Slower	When you need to check object properties

Pure vs Impure

Aspect	Pure (Default)	Impure
Performance	✅ Fast (cached)	⚠️ Slow (runs every cycle)
Recomputation	Only when input reference changes	Every change detection cycle
Use Case	✅ Default, recommended	⚠️ Rare (deep object checking)
Best Practice	✅ Always use pure unless needed	❌ Avoid unless absolutely necessary

Best Practice: Always use **pure pipes** (default). Only use impure pipes when you need to detect changes inside objects/arrays that don't change reference.

Built-in Pipes

Common Pipes

Pipe	Purpose	Example
`date`	Format dates	`{{ date | date:'short' }}`
`currency`	Format currency	`{{ price | currency:'USD' }}`
`number`	Format numbers	`{{ number | number:'1.2-2' }}`
`percent`	Format percentages	`{{ value | percent }}`
`uppercase`	Convert to uppercase	`{{ text | uppercase }}`
`lowercase`	Convert to lowercase	`{{ text | lowercase }}`
`titlecase`	Convert to title case	`{{ text | titlecase }}`
`json`	Format as JSON	`{{ object | json }}`
`slice`	Slice arrays/strings	`{{ array | slice:0:5 }}`

`async`	Subscribe to observables	`{{ data$ | async }}`

Performance Optimization

Optimization Techniques

Technique	Impact	Difficulty	When to Use
OnPush Strategy	High	Low	Most components
TrackBy + async pipe	Medium	Low	*ngFor with lists
Lazy Loading	High	Medium	Routes, modules
Caching	High	Medium	API responses, computations
Signal Emission	Medium	Low	Angular 16+
Virtual Scrolling	High	Medium	Long lists

Priority Order:

1. OnPush + TrackBy + Async Pipe
2. Lazy Loading
3. Bundle Optimization
4. Everything else

Quick Example: List + Async Pipe + @if / @for (BehaviorSubject)

Component (BehaviorSubject + mocked list):

```
interface Person {
  id: number;
  name: string;
  age: number;
}

const MOCK_PEOPLE: Person[] = [
  { id: 1, name: "Alice", age: 28 },
  { id: 2, name: "Bob", age: 32 },
  { id: 3, name: "Carol", age: 24 },
];

export class PersonListComponent {
  private peopleSubject = new BehaviorSubject<Person[]>(MOCK_PEOPLE);
  people$ = this.peopleSubject.asObservable();

  constructor() {
    this.loadPeople();
  }

  loadPeople(): void {
    this.peopleSubject.next(MOCK_PEOPLE);
  }
```

```
  /** Update one entry by id. Emits a new array so the async pipe + @for re-render. */
  updatePerson(id: number, updates: Partial<Person>): void {
    const current = this.peopleSubject.getValue();
    const index = current.findIndex((p) => p.id === id);
    const updated = [...current];
    if (index === -1) return;
    updated[index] = { ...current[index], ...updates };
    this.peopleSubject.next(updated);
  }

  /** Add a new entry. id = length + 1 (use max(id)+1 if you also delete items). */
  addPerson(person: Omit<Person, "id">): void {
    const current = this.peopleSubject.getValue();
    const nextId = current.length + 1;
    this.peopleSubject.next([...current, { ...person, id: nextId }]);
  }

  /** Remove one entry by id. */
  deletePerson(id: number): void {
    const current = this.peopleSubject.getValue();
    this.peopleSubject.next(current.filter((p) => p.id !== id));
  }
}
```

Template (async pipe with `as` + @if / @for + create / edit / delete):

```
@if (people$ | async; as people) {
  <button (click)="addPerson({ name: 'New Person', age: 0 })">Add</button>
  @for (person of people; track person.id) {
    <div>
      {{ person.id }} — {{ person.name }}, age {{ person.age }}
      <button (click)="updatePerson(person.id, { name: 'Updated', age: 99 })">Edit</button>
      <button (click)="deletePerson(person.id)">Delete</button>
    </div>
  }
}
```

RxJS

Concept

library for dealing with asynchronous and event-based logic using observable sequences.

Core Concepts

Concept	What It Is	Purpose	Most Used For
Observable	Stream of values over time	Represents async data stream	HTTP requests, user events
Observer	Subscribes to observables	Receives values from observable	`.subscribe()` method
Pipe	Method to chain operators	Combine multiple operators	`observable$.pipe(map(), filter())`
Subject	Observable + Observer (hot)	Multicast values to subscribers	Shared state, event bus
BehaviorSubject	Subject that stores last value	Shared state with initial value	State management in services
Subscription	Represents execution of observable	Manage and cancel subscriptions	Unsubscribe to prevent leaks

Most Used Operators

Category	Operator	Purpose	Use Case
Transform	`map`	Transform each value	Convert data format
Transform	`switchMap`	Switch to new observable	Search, route params
Transform	`mergeMap`	Merge all inner observables	Multiple parallel requests
Filter	`filter`	Filter values by condition	Conditional data
Filter	`takeUntil`	Take until another emits	Unsubscription pattern
Filter	`debounceTime`	Wait for silence before emitting	User input, search
Combine	`combineLatest`	Combine latest from each	Multiple data sources
Combine	`forkJoin`	Wait for all to complete	Multiple HTTP requests
Error	`catchError`	Handle errors gracefully	Error handling
Utility	`shareReplay`	Cache and replay values	HTTP request caching

map vs switchMap

Operator	Input Type	Output Type	Behavior	Example	What Happens in Example
map	Value	Transformed value	Transforms value synchronously	`users$.pipe(map(u => u.name))`	User object → user name (string)
switchMap	Value	Observable	Cancels previous, switches to new	`users$.pipe(switchMap(u => http.get(u.id)))`	User object → HTTP request (cancels previous if new user arrives)

Cold vs Hot Observables

Type	Execution Type	Example	How to Make Hot
Cold	New execution each	HTTP request per subscribe	Use `share()` or `shareReplay(1)`
Hot	Shared execution	One HTTP for all subscribers	Already hot (Subjects, DOM events)

share() vs shareReplay()

Operator	What It Does	Caching	In Simple Terms	Use Case
share()	Makes observable hot, shares execution	❌ No	Everyone listens live, but late subscribers miss past values	Share execution only
shareReplay(N)	Makes observable hot + caches last N values	✅ Yes	Everyone listens live, and late subscribers get the last N values	HTTP caching, late subscribers

SPA (Single Page Application)

Concept

Web application that loads a single HTML page and updates dynamically without reloading.

- Navigation without reload (using Router)
- Better UX (smooth transitions)
- Better Performance (fewer requests)

How Angular Implements

Router with `router-outlet`, components loaded dynamically.

HTTP Client

Concept

Angular's HTTP Client is a service for making HTTP requests. It's built on top of XMLHttpRequest (not fetch) and returns Observables, making it perfect for reactive programming with RxJS.

Key Features

Feature	Description	Benefit
Observable-based	Returns Observables	Works with RxJS operators
Type-safe	TypeScript support	Type safety for requests/responses
Interceptors	Request/response middleware	Centralized logic
Automatic JSON	Parses JSON automatically	Less boilerplate
Error Handling	Built-in error handling	Better error management

HTTP Methods

Method	Purpose	When to Use
GET	Retrieve data	Fetching resources
POST	Create new resource	Creating new entities
PUT	Replace resource	Full update of existing entity
PATCH	Partial update	Update specific fields
DELETE	Remove resource	Deleting entities

HTTP Options

Option	Purpose	Example
headers	Custom request headers	Authorization, Content-Type
params	Query parameters	Pagination, filters
responseType	Response format	json, text, blob, arraybuffer

Interceptors

Aspect	Description
Purpose	Intercept and modify HTTP requests/responses
Execution	Run in sequence (order matters)

Use Cases	Authentication, logging, error handling, caching
Registration	Must use `multi: true` for multiple interceptors

Error Handling

Error Type	Status Code Range	Description
Network	0	Connection failed
Client Error	400-499	Bad request, not found
Server Error	500-599	Server-side issues

Best Practices

Practice	Why
Type Requests/Responses	Type safety, better DX
Handle Errors	Proper error management
Use Interceptors	Centralized logic, DRY
Use Async Pipe	Automatic unsubscribe
Cache When Appropriate	Performance optimization

HTTP Client vs Fetch

Aspect	HTTP Client	Fetch
Returns	Observable	Promise
Built on	XMLHttpRequest	Fetch API
RxJS Integration	✅ Native	❌ Needs conversion
Interceptors	✅ Built-in	❌ Manual
Change Detection	✅ Automatic	❌ Manual

Storage (Cookies, LocalStorage, SessionStorage)

Concept

Web storage provides different mechanisms for storing data in the browser. Each storage type has different characteristics, use cases, and limitations.

Storage Types Comparison

Feature	Cookie	LocalStorage	SessionStorage
Capacity	~4KB	~5-10MB	~5-10MB
Persistence	Until expiration	Forever	Until tab closes
Server Access	✅ Yes (sent with requests)	❌ No	❌ No
Client Access	✅ Yes	✅ Yes	✅ Yes
Scope	Domain-wide	Domain-wide	Tab-specific
Use Case	Tokens, small data	App state, preferences	Form data, temp data

When to Use

Storage Type	When to Use	Why
Cookie	JWT tokens, small config data	Sent to server, secure with httpOnly
LocalStorage	User preferences, app state, cache	Persists across sessions
SessionStorage	Form data, temporary state, cart	Cleared on tab close, tab-specific

Security Considerations

Storage Type	Security Risk	Mitigation
Cookies	XSS, CSRF	httpOnly, secure, sameSite
LocalStorage	XSS	Sanitize input, CSP
SessionStorage	XSS	Sanitize input, CSP

Best Practices

Practice	Why
Handle Errors	Quota exceeded, storage disabled
Use JSON for Complex	Only strings stored
Don't Store Sensitive	Security risk

Check Availability	Storage may be disabled
Clear Old Data	Prevent storage bloat

WebSockets

Concept

WebSockets provide a persistent, full-duplex communication channel between client and server. Unlike HTTP (request-response), WebSockets allow real-time bidirectional communication.

WebSockets vs HTTP

Aspect	HTTP	WebSockets
Connection	Request-Response	Persistent
Direction	One-way (client → server)	Bidirectional
Overhead	High (headers each request)	Low (after handshake)
Real-time	❌ No (polling needed)	✅ Yes
Use Case	Standard API calls	Real-time updates

When to Use WebSockets

Scenario	Use WebSockets?	Why
Real-time chat	✅ Yes	Instant messaging
Live notifications	✅ Yes	Push notifications
Collaborative editing	✅ Yes	Real-time sync
Live dashboards	✅ Yes	Real-time data
Standard CRUD	❌ No	HTTP is sufficient
One-time data fetch	❌ No	HTTP is simpler

WebSocket States

State	Value	Description
`CONNECTING`	0	Connection is being established
`OPEN`	1	Connection is open and ready
`CLOSING`	2	Connection is being closed
`CLOSED`	3	Connection is closed

Best Practices

Practice	Why

Handle Reconnection	Connections can drop
Check Ready State	Only send when OPEN
Parse Messages Safely	Invalid JSON handling
Clean Up on Destroy	Prevent memory leaks
Use WSS in Production	Secure connection
Authenticate Connection	Security

CORS (Cross-Origin Resource Sharing)

Concept

CORS is a security check that allows / restricts web pages from making requests to a different domain than the one serving the web page.

Same-Origin Policy

Aspect	Same Origin?	Why
Protocol	Must match	http vs https = different
Domain	Must match	example.com vs api.com = different
Port	Must match	:80 vs :8080 = different

CORS Headers

Header	Direction	Purpose
`Access-Control-Allow-Origin`	Response	Allowed origins
`Access-Control-Allow-Methods`	Response	Allowed HTTP methods
`Access-Control-Allow-Headers`	Response	Allowed headers
`Access-Control-Allow-Credentials`	Response	Allow credentials
`Origin`	Request	Request origin

Testing

Concept

Testing in Angular ensures code quality, prevents regressions, and provides confidence when refactoring. Angular provides excellent testing tools and frameworks for unit, integration, and end-to-end testing.

Testing Types

Type	Scope	Tools	Purpose
Unit Tests	Individual components/services	Jasmine, Jest	Test isolated units
Integration Tests	Component interactions	Jasmine, Jest	Test component integration
E2E Tests	Full application flow	Cypress, Playwright	Test user workflows

Testing Tools

Tool	Type	Use Case
Jest	Unit/Integration	Fast, modern, recommended
Jasmine	Unit/Integration	Default Angular testing
Karma	Test Runner	Runs tests in browsers
Cypress	E2E	End-to-end testing
Playwright	E2E	Cross-browser E2E testing

Testing Concepts

Concept	Description	Purpose
Test Bed	Angular testing environment	Configure testing module
Fixtures	Component wrapper for testing	Access component and DOM
Spies	Mock functions for testing	Verify function calls
Mocks	Fake implementations	Isolate units under test
Matchers	Assertion functions	Verify expectations

Code Coverage Goals

Metric	Target	Good
Statements	> 80%	> 90%
Branches	> 75%	> 85%

Functions	> 80%	> 90%
Lines	> 80%	> 90%

Best Practices

Practice	Why
Test Behavior, Not Implementation	Tests survive refactoring
Use Descriptive Names	Clear test intent
Arrange-Act-Assert Pattern	Clear test structure
Mock External Dependencies	Isolate units
Test Edge Cases	Handle null, undefined, empty
Maintain Good Coverage	Confidence in code quality

Common Mistakes

Mistake	Solution
Testing Implementation	Test behavior instead
Not Mocking Dependencies	Always mock HTTP, Router, etc.
Not Cleaning Up	Clean up after tests
Not Testing Error Cases	Test success and failure paths
Testing Framework Code	Test your code, not Angular's

HTTP vs HTTPS

Concept

HTTP (Hypertext Transfer Protocol) and HTTPS (HTTP Secure) are protocols for transferring data over the web.

HTTPS is HTTP with encryption (TLS/SSL), providing security and data integrity.

HTTP vs HTTPS

Aspect	HTTP	HTTPS
Port	80	443
Encryption	❌ No (plain text)	✅ Yes (TLS/SSL encryption)
Security	Vulnerable to interception	Secure, encrypted data
Certificate	Not required	Requires SSL/TLS certificate
SEO	Lower ranking	Higher ranking (Google preference)
Browser Trust	Shows "Not Secure" warning	Shows lock icon, trusted
Performance	Slightly faster	Slightly slower (encryption overhead)
Use Case	Internal networks, testing	Production, sensitive data, e-commerce

When to Use HTTP vs HTTPS

Scenario	Protocol	Reason
Production	HTTPS	Security, SEO, user trust
Development	HTTP	Easier setup, no cert needed
API with tokens	HTTPS	Protect sensitive credentials
E-commerce	HTTPS	Required for payment processing
Internal network	HTTP	Trusted network, faster
Public website	HTTPS	Browser warnings, SEO

CDN (Content Delivery Network)

Concept

CDN is a network of distributed servers that deliver web content to users based on their geographic location, reducing latency and improving performance by serving content from the nearest server.

How CDN Works

Step	Process	Benefit
1	User requests content	Initial request
2	CDN routes to nearest edge server	Reduced latency
3	Edge server checks cache	Fast response if cached
4	If not cached, fetches from origin	First request slower
5	Caches content for future requests	Subsequent requests fast
6	Delivers to user	Low latency delivery

CDN Benefits

Benefit	Description	Impact
Performance	Faster load times	Better UX, lower bounce
Scalability	Handles traffic spikes	No server overload
Reliability	Redundant servers	High availability
Bandwidth Savings	Offloads traffic from origin	Lower server costs
Global Reach	Serves users worldwide	Consistent performance
Security	DDoS protection, SSL/TLS	Enhanced security

Signals

Concept

Signals are reactive primitives (Angular 16+) that automatically track dependencies and update the UI when values change. They provide granular change detection and eliminate the need for manual subscriptions.

Declaring Signals

Type	Declaration	Purpose
signal	`count = signal(0)`	Writable reactive value
computed	`double = computed(() => count() * 2)`	Derived value from signals
effect	`effect(() => console.log(count()))`	Side effect on changes

Propagating Signals

Method	Purpose	Example
set()	Replace value	`count.set(10)`
update()	Update based on current value	`count.update(v => v + 1)`
()	Read value	`count()` returns value

Simple Examples

Basic Signal

```
// Component
count = signal(0);

increment() {
  this.count.update(v => v + 1);
}
```

```
<!-- Template -->
<p>Count: {{ count() }}</p>
<button (click)="increment()">+</button>
```

Computed Signal

```
// Component
price = signal(10);
quantity = signal(2);
total = computed(() => this.price() * this.quantity());
```

```
<!-- Template -->
<p>Total: {{ total() }}</p>
```

Signal in Service (Shared State)

```
// Service
private usersState = signal<User[]>([]);
users = this.usersState.asReadonly();

loadUsers() {
  this.http.get<User[]>('/api/users').subscribe(users => {
    this.usersState.set(users);
  });
}
```

```
// Component
users = this.userService.users;

// Template automatically updates when users change
```

Guards

Concept

Guards are interfaces that control navigation to and from routes. They determine whether a route can be activated, deactivated, or loaded, providing route protection and access control.

Key Point: Guards are executed before route activation, allowing you to prevent unauthorized access, redirect users, or cancel navigation based on conditions like authentication, permissions, or unsaved changes.

Guard Types

Guard Type	Interface	Purpose	Return Type
CanActivate	`CanActivate`	Control route activation	`boolean | UrlTree | Observable`
CanActivateChild	`CanActivateChild`	Control child route activation	`boolean | UrlTree | Observable`
CanDeactivate	`CanDeactivate`	Control route deactivation (unsaved data)	`boolean | Observable`
CanLoad	`CanLoad`	Control lazy-loaded module loading	`boolean | Observable`
CanMatch	`CanMatch`	Conditionally match routes (Angular 14+)	`boolean | Observable`

When to Use

- **CanActivate**: Authentication, authorization checks before route access
- **CanDeactivate**: Prevent navigation when form has unsaved changes
- **CanLoad**: Prevent loading of lazy modules if user lacks permission
- **CanMatch**: Conditionally match routes based on runtime conditions

Example

```
// Auth Guard
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isAuthenticated()) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}
```

```
// Route configuration
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [AuthGuard]
}
```

Practical Example: BehaviorSubject with Inline Templates

Overview

This practical example demonstrates how to use **BehaviorSubject** from RxJS with **inline templates**, **async pipe**, and structural directives (***ngIf** and ***ngFor**) in an Angular component.

Scenario

We'll build a simple user list component that:

- Uses BehaviorSubject to manage user data state
- Displays users using *ngFor with async pipe
- Allows adding new users
- Allows editing a user
- Allows deleting users
- All HTML is declared inline within the component file

Complete Component Example

```
import { Component, OnInit, OnDestroy } from "@angular/core";
import { CommonModule } from "@angular/common";
import { BehaviorSubject, Observable, Subject } from "rxjs";

interface User {
  id: number;
  name: string;
  email: string;
}

@Component({
  selector: "app-user-list",
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="user-list-container">
      <h2>User Management</h2>

      <div class="user-header">
        <button (click)="addUser()">Add User</button>
      </div>

      <!-- User List with *ngIf and *ngFor using async pipe -->
      <div *ngIf="users$ | async as users">
        <div class="user-list">
          <div
            *ngFor="let user of users; trackBy: trackByUserId"
            class="user-item"
```

```
          >
            <div class="user-info">
              <h4>{{ user.name }}</h4>
              <p>{{ user.email }}</p>
            </div>
            <button (click)="deleteUser(user.id)">Delete</button>
          </div>
        </div>
      </div>
    </div>
  `,
})
export class UserListComponent implements OnInit, OnDestroy {
  // BehaviorSubject to manage user data state
  private usersSubject = new BehaviorSubject<User[]>([]);

  // Public observable exposed to template
  public users$: Observable<User[]> = this.usersSubject.asObservable();

  // Destroy subject for cleanup (use with takeUntil in any manual subscriptions)
  private destroy$ = new Subject<void>();

  private nextUserId = 1;

  ngOnInit(): void {
    // Initialize with some users
    this.usersSubject.next([
      { id: 1, name: "John Doe", email: "john@example.com" },
      { id: 2, name: "Jane Smith", email: "jane@example.com" },
    ]);
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }

  /**
   * Add a new user to the list
   */
  addUser(): void {
    const currentUsers = this.usersSubject.getValue();
    const newUser: User = {
      id: this.nextUserId++,
      name: `User ${this.nextUserId}`,
      email: `user${this.nextUserId}@example.com`,
    };

    // Update BehaviorSubject with new array
    this.usersSubject.next([...currentUsers, newUser]);
  }
```

```
  /**
   * Update one user by id. Emits a new array so the async pipe re-renders.
   */
  updateUser(id: number, updates: Partial<User>): void {
    const current = this.usersSubject.getValue();
    const index = current.findIndex((u) => u.id === id);
    const updated = [...current];
    if (index === -1) return;
    updated[index] = { ...current[index], ...updates };
    this.usersSubject.next(updated);
  }

  /**
   * Delete a user from the list
   */
  deleteUser(userId: number): void {
    const currentUsers = this.usersSubject.getValue();
    const filteredUsers = currentUsers.filter((user) => user.id !== userId);
    this.usersSubject.next(filteredUsers);
  }

  /**
   * TrackBy function for *ngFor performance optimization
   */
  trackByUserId(index: number, user: User): number {
    return user.id;
  }
}
```

Hoisting

Concept

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before code execution. Only declarations are hoisted, not initializations.

Key Point: Variables declared with `var` are hoisted and initialized with `undefined`, while `let` and `const` are hoisted but remain in a "temporal dead zone" until their declaration line.

Hoisting Behavior

Declaration Type	Hoisted?	Initial Value	Temporal Dead Zone
`var`	Yes	`undefined`	No
`let`	Yes	Uninitialized	Yes
`const`	Yes	Uninitialized	Yes
`function`	Yes	Function body	No
Arrow function (var)	Variable	`undefined`	No
Arrow function (let/const)	Variable	Uninitialized	Yes
`class`	No	-	Yes

When Hoisting Happens

- **Function declarations**: Fully hoisted (can be called before declaration)
- **Variable declarations**: Hoisted but initialized as `undefined` (var) or uninitialized (let/const)
- **Function expressions & Arrow functions**: Only the variable is hoisted, not the function itself

Example

```
// var hoisting
console.log(x); // undefined (not error)
var x = 5;

// let/const hoisting
console.log(y); // ReferenceError: Cannot access before initialization
let y = 10;

// Function hoisting
sayHello(); // Works! "Hello"
function sayHello() {
  console.log("Hello");
}

// Arrow function hoisting
```

```
sayHi(); // TypeError: sayHi is not a function (var) or ReferenceError (let/const)
var sayHi = () => console.log("Hi");
```

Closures & Scope

Concept

A closure is a function that has access to variables in its outer scope, even after the outer function has returned. Scope determines the accessibility of variables, functions, and objects in different parts of your code.

Key Point: Closures allow inner functions to "remember" and access variables from their outer scope, creating private variables and enabling powerful patterns like function factories and module patterns.

Scope Types

Scope Type	Description	Access Level
Global	Variables declared outside functions	Accessible everywhere
Function	Variables declared inside functions	Only within function
Block	Variables in `{}` blocks (let/const)	Only within block
Lexical	Determined by where code is written	Inner functions access outer

Scope Chain & Closure

Scope chain searches inner to outer scopes. Closures allow inner functions to access outer variables even after outer function returns.

Example

```
// Lexical Scope & Closure
function outer() {
  const name = "Vitor";
  return function inner() {
    console.log(name); // Accesses outer scope
  };
}

const closure = outer();
closure(); // "Vitor"

// Closure with private variable
function counter() {
  let count = 0;
  return () => ++count;
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
```

This Keyword

Concept

`this` is a reference to the object that is executing the current function. Its value depends on how the function is called, not where it's defined.

Key Point: `this` is determined at runtime based on the call site, not the definition site. Arrow functions don't have their own `this` - they inherit it from the enclosing scope.

This Binding Rules

Context	`this` refers to	Example
Global	`window` (browser) or `global` (Node)	`this` in global scope
Method call	Object calling the method	`obj.method()`
Constructor	New instance being created	`new MyClass()`
Arrow function	Lexical `this` from outer scope	`() => this`
Event handler	Element that triggered event	`button.onclick`
Explicit binding	First argument of call/apply/bind	`func.call(obj)`

Ways to Control This

- **`.call()`** : Calls function with explicit `this` and arguments
- **`.apply()`** : Calls function with explicit `this` and array of arguments
- **`.bind()`** : Returns new function with bound `this`

Example

```
const person = {
  name: "Vitor",
  greet: function () {
    return `Hello, ${this.name}`;
  },
};
console.log(person.greet()); // "Hello, Vitor"

// .call() - explicit this + arguments
function introduce(city, country) {
  return `${this.name} from ${city}, ${country}`;
}
console.log(introduce.call(person, "São Paulo", "Brazil"));

// .apply() - explicit this + array of arguments
console.log(introduce.apply(person, ["Rio", "Brazil"]));
```

```
// .bind() - returns new function with bound this
const boundGreet = person.greet.bind(person);
console.log(boundGreet()); // "Hello, Vitor"
```

Value vs Reference

Concept

JavaScript has two types of data: primitives (passed by value) and objects (passed by reference).

Primitives are copied by value, creating independent copies. Objects are copied by reference, meaning multiple variables can point to the same object in memory.

Data Types

Type	Category	Passed By	Behavior on Assignment
`number`	Primitive	Value	Creates independent copy
`string`	Primitive	Value	Creates independent copy
`boolean`	Primitive	Value	Creates independent copy
`null`	Primitive	Value	Creates independent copy
`undefined`	Primitive	Value	Creates independent copy

| `object` | Reference | Reference | Points to same object | | `array` | Reference | Reference | Points to same object | | `function` | Reference | Reference | Points to same object |

Key Differences

- **Primitives**: Immutable, copied when assigned or passed
- **Objects**: Mutable, shared reference when assigned or passed
- **Comparison**: Primitives compare by value, objects compare by reference

Example

```
// Primitives (by value)
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 (unchanged)

// Objects (by reference)
let obj1 = { name: "Vitor" };
let obj2 = obj1;
obj2.name = "John";
console.log(obj1.name); // "John" (changed!)

// Arrays (by reference)
let arr1 = [1, 2, 3];
let arr2 = arr1;
arr2.push(4);
console.log(arr1); // [1, 2, 3, 4] (changed!)
```

Deep Copy vs Shallow Copy

Concept

A shallow copy creates a new object but copies only the top-level properties.

Nested objects/arrays still reference the original.

A deep copy creates a completely independent copy, including all nested objects and arrays.

Copy Methods Comparison

Method	Type	Nested Objects	Performance	Use Case
`Object.assign()`	Shallow	Shared	Fast	Simple object copy
Spread operator `{...}`	Shallow	Shared	Fast	Simple object copy
`Array.slice()`	Shallow	Shared	Fast	Array copy
`JSON.parse(JSON.stringify())`	Deep	Independent	Slow	Simple deep copy
`structuredClone()`	Deep	Independent	Medium	Modern deep copy
`Lodash.cloneDeep()`	Deep	Independent	Medium	Complex deep copy

When to Use

- **Shallow copy**: When you only need top-level properties or nested objects won't change
- **Deep copy**: When you need complete independence, especially with nested structures

Limitations

- `JSON.parse(JSON.stringify())` loses functions, `undefined`, `Symbol`, and `Date` objects
- `structuredClone()` doesn't work with functions, classes, or certain built-in objects

Example

```
const original = {
  name: "Vitor",
  address: { city: "São Paulo", country: "Brazil" },
};

// Shallow copy
const shallow = { ...original };
shallow.address.city = "Rio";
console.log(original.address.city); // "Rio" (changed!)

// Deep copy
const deep = JSON.parse(JSON.stringify(original));
deep.address.city = "Brasília";
console.log(original.address.city); // "Rio" (unchanged)
```

```
// Modern deep copy
const deep2 = structuredClone(original);
deep2.address.city = "Salvador";
console.log(original.address.city); // "Rio"
```

Event Loop

Concept

The event loop is JavaScript's mechanism for handling asynchronous operations. It manages the execution of code, callbacks, and events in a single-threaded environment, ensuring non-blocking behavior.

Key Point: JavaScript is single-threaded but achieves concurrency through the event loop, which processes tasks from the call stack, callback queue, and microtask queue in a specific order.

Event Loop Phases

Phase	Description	Priority
Call Stack	Synchronous code execution	Highest
Microtasks	Promises, queueMicrotask, MutationObserver	High
Macrotasks	setTimeout, setInterval, I/O, events	Low

Execution Order

1. Execute all synchronous code from call stack
2. Process all microtasks (until queue is empty)
3. Process one macrotask
4. Repeat from step 2

Task Types

- **Macrotasks**: setTimeout, setInterval, I/O operations, UI rendering, event handlers
- **Microtasks**: Promise.then/catch/finally, queueMicrotask(), MutationObserver

Example

```
console.log("1"); // Synchronous

setTimeout(() => console.log("2"), 0); // Macrotask

Promise.resolve().then(() => console.log("3")); // Microtask

console.log("4"); // Synchronous

// Output: 1, 4, 3, 2
// Explanation: Sync code runs first, then microtasks, then macrotasks
```

Asynchronous Programming

Concept

Asynchronous programming allows code to run without blocking the main thread, enabling operations like API calls, file I/O, and timers to execute in the background while other code continues running.

Key Point: JavaScript handles asynchrony through callbacks, Promises, and async/await. The event loop manages execution order, with microtasks (Promises) having priority over macrotasks (setTimeout).

Async Patterns

Pattern	Syntax	Pros	Cons
Callbacks	`func(callback)`	Simple, universal	Callback hell, no error handling
Promises	`.then()/.catch()`	Chainable, better error handling	Can still be verbose
async/await	`async/await`	Clean, synchronous-like syntax	Requires async function

Promise States

- **Pending**: Initial state, neither fulfilled nor rejected
- **Fulfilled**: Operation completed successfully
- **Rejected**: Operation failed

Event Loop & Microtasks

Promises use microtask queue (higher priority than macrotasks like setTimeout).

Example

```javascript
// Callback
setTimeout(() => console.log("Callback"), 1000);

// Promise
fetch("/api/data")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error));

// async/await
async function fetchData() {
  try {
    const response = await fetch("/api/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
```

```
  }
}
```

DOM Manipulation

Concept

DOM (Document Object Model) manipulation involves accessing, modifying, and interacting with HTML elements using JavaScript. Modern APIs provide efficient ways to traverse, modify, and listen to DOM events.

Key Point: Use modern DOM APIs (querySelector, addEventListener) over legacy methods (getElementById, onclick). Understand event delegation for better performance with dynamic content.

DOM Selection Methods

Method	Returns	Performance	Use Case
`querySelector()`	First match	Fast	Single element
`querySelectorAll()`	NodeList	Fast	Multiple elements
`getElementById()`	Element	Fastest	Single element by ID
`getElementsByClassName()`	HTMLCollection	Fast	Multiple by class

Event Handling

Method	Description	Event Bubbling	Use Case
`addEventListener()`	Modern event listener	Yes (default)	Recommended approach
`onclick` property	Legacy inline handler	Yes	Avoid in new code
Event delegation	Listen on parent, filter target	Yes	Dynamic content

Modern DOM APIs

API	Purpose	Use Case
Event delegation	Listen on parent, filter by target	Dynamic content, performance
MutationObserver	Observe DOM changes	Detect element modifications
IntersectionObserver	Observe element visibility	Lazy loading, infinite scroll
requestAnimationFrame	Sync updates with browser repaint	Animations, smooth updates

Example

```
// Selection & Event handling
const button = document.querySelector("#myButton");
button.addEventListener("click", (e) => {
  console.log("Clicked!", e.target);
});
```

```
// Event delegation
document.addEventListener("click", (e) => {
  if (e.target.matches(".dynamic-item")) {
    console.log("Dynamic item clicked");
  }
});
```